BK-AHR: Bayesian Kronecker-Preconditioned Adaptive Hybrid Routing for Noise-Robust Data-Free Test-Time Model Merging

Anonymous Author [*1]

Abstract

Test-Time Model Merging (TTMM) dynamically combines task-specific expert neural networks in weight space on-the-fly to handle non-stationary, unlabeled test streams. However, existing open-world TTMM frameworks face severe limitations: they rely on Euclidean distance-based prototype spaces which suffer from scale distortion under noise, or they employ spherical normalization which triggers representational collapse on sparse background domains under severe environmental perturbations. Furthermore, they violate the strict data-free test-time assumption by requiring offline calibration data for parameter sensitivity preconditioning. To resolve these core bottlenecks, we propose BK-AHR (Bayesian Kronecker-Preconditioned Adaptive Hybrid Routing), a fully data-free, unsupervised, and noise-robust TTMM framework. BK-AHR introduces: (1) a denoised, sparsity-aware gating mechanism based on thresholded Hoyer's sparsity of the incoming stream to dynamically select the optimal routing domain (Euclidean vs. scale-invariant Angular); (2) a differentiable, on-the-fly Kronecker sensitivity estimation using parameter gradients to precondition updates and coherence regularizers; and (3) a moment-matched soft Bayesian Mixture-of-Gaussians Batch Normalization buffer fusion. Our extensive empirical evaluations on non-stationary vision streams demonstrate that BK-AHR successfully resolves the sparsity-density trade-off, outperforming unnormalized baselines by achieving a highly robust and stable performance under severe environmental corruptions, without requiring any offline source task calibration data.

1. Introduction

The rapid progress of deep learning has shifted the paradigm from training massive models from scratch toward fine-tuning foundation models on specialized target domains, yielding vast libraries of specialized expert networks (Wortsman et al., 2022). Integrating these expert networks into a single, cohesive multi-task architecture without incurring prohibitive retraining costs has motivated the development of model merging and weight-space averaging techniques (Ilharco et al., 2023; Yadav et al., 2023; Jin et al., 2023). By interpolating parameters directly, model merging bypasses the need for joint training data, maintaining parameter efficiency while achieving remarkable multi-task capabilities.

Recently, this concept has been extended to the streaming inference phase via Test-Time Model Merging (TTMM) (Yang et al., 2024; Hubotter et al., 2026). In practical deployments, neural networks encounter non-stationary target streams where the active task distribution shifts continuously over time. TTMM dynamically interpolates the weight parameters of specialized expert networks on-the-fly to match the active task distribution of an unlabeled, non-stationary test stream:

$$\bar{\theta}_t = \lambda(t)\theta_1 + (1 - \lambda(t))\theta_2 \tag{1}$$

where θ_1 and θ_2 represent the weights of two expert networks, and $\lambda(t) \in [0, 1]$ represents the dynamic merging coefficient at time step t.

Despite initial successes, adapting merging coefficients under realistic, open-world deployment remains a fundamental challenge. State-of-the-art open-world TTMM frameworks utilize feature prototype routing to detect novel domains and compute merging coefficients. However, we identify three critical, previously unaddressed bottlenecks in existing literature:

[1]Department of Computer Science, Anonymous Institution, Location, Country. Correspondence to: Anonymous Author <anon@institution.edu>.

Proceedings of the 43^{rd} International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026.

1. The Sparsity-Density Gap Under Noise: Euclidean distance-based prototype spaces suffer from scale distortion under noise, compressing distance differences and causing fixed temperature routing to output flat, uniform coefficients, which triggers representational collapse. While Contrastive Prototypes with Angular Margin (CP-AM) reformulates routing in an angular, cosine domain to establish scale invariance, it reveals a fundamental trade-off: background sparsity (as in MNIST) exposes a vulnerability to spherical normalization under severe additive pixel-wise noise, whereas textured, dense domains (as in FashionMNIST) show massive robustness gains.

2. Environmental Noise Masking physical Sparsity: An adaptive hybrid routing (AHR) that dynamically estimates feature sparsity to alternate between Euclidean and Angular routing fails under environmental noise. Added noise fills the sparse background with high-frequency perturbations, tricking the sparsity estimator into classifying noisy sparse tasks as dense, which triggers incorrect spherical normalization routing and destroys model utility.

3. Strict Data-Free Violation: State-of-the-art layer-wise adaptation methods (like CLW-Fisher) require pre-computing diagonal Fisher Information matrices on clean offline calibration datasets. This directly violates the strict data-free test-time assumption because source datasets are often private, proprietary, or inaccessible at deployment time.

To resolve these fundamental bottlenecks, we propose BK-AHR (Bayesian Kronecker-Preconditioned Adaptive Hybrid Routing), a fully data-free, unsupervised, and noise-robust TTMM framework. BK-AHR completely eliminates offline calibration data dependencies and resolves the sparsity-density trade-off. Our core contributions are:

- We introduce a Denoised Sparsity-Aware Gating (AHR) mechanism based on thresholded Hoyer's sparsity of the incoming stream. By zeroing out noise-level fluctuations before sparsity estimation, our gating mechanism correctly identifies the underlying physical domain sparsity under severe noise, allowing the system to dynamically alternate between robust Euclidean routing on sparse tasks and scale-invariant Angular routing on dense tasks.
- We derive a fully differentiable, hook-free On-the-Fly Kronecker Preconditioning and Adaptive Consensus Coherence Regularization method. By leveraging PyTorch's stateless functional_call and computing sensitivities directly from parameter gradients, our method maintains stable gradient flow and restricts sensitive layers from drifting destructively, completely bypassing the need for offline Fisher matrices.
- We integrate Soft Bayesian MoG BN Statistic Fusion which blends Batch Normalization running statistics via mathematically exact moment-matching under a Mixture-of-Gaussians assumption, preventing representational distortion.
- Our empirical evaluations on non-stationary vision streams show that BK-AHR achieves outstanding robust performance, outperforming unnormalized TTA baselines on clean and noisy segments while preserving novel domain safety on KMNIST.

2. Related Work

Model Merging and Weight-Space Averaging: Weight-space model merging represents a parameter-efficient alternative to prediction-level Mixtures-of-Experts (MoE) or ensembles. Static merging methods like Model Soups (Wortsman et al., 2022), Task Arithmetic (Ilharco et al., 2023), TIES-Merging (Yadav et al., 2023), RegMean (Jin et al., 2023), and Git Re-basin (Ainsworth et al., 2023) consolidate specialized expert skills without joint training data, and are often applied to parameter-efficient fine-tuning (PEFT) increments like low-rank adapters (LoRA) (Hu et al., 2021) to merge task-specific abilities. Fisher-weighted averaging (Matena & Raffel, 2022) and REPAIR (Jordan et al., 2022) scale parameter merging based on curvature and renormalize activation distributions to address representational mismatches in merged models. However, these static methods cannot adapt dynamically to non-stationary, streaming covariate shifts at deployment time.

Test-Time Model Merging and Adaptation: Moving weight-space model merging to online deployment, Test-Time Model Merging (TTMM) dynamically solves for optimal merging coefficients on-the-fly based on unsupervised objectives (Yang et al., 2024). Unlike full Test-Time Adaptation (TTA) methods like TENT (Wang et al., 2021) or EATA (Niu et al., 2022) which update model parameters via backpropagation, TTMM restricts parameter search to weight interpolations, avoiding catastrophic representation collapse. Modern TTMM baselines like CLW-Fisher (Hubotter et al., 2026) and BK-CoMerge introduce prior-guided initialization and temperature scaling to handle task boundaries and environmental corruptions.

Angular Margin Learning: In deep representation

learning, standard softmax classifiers often suffer from poor discriminative features. To address this, angular margin losses like CosFace (Wang et al., 2018) and ArcFace (Deng et al., 2019) introduce an additive angular/cosine margin during pre-training. By normalizing both weight vectors and feature representations, these losses force the network to minimize within-class angular variance, creating highly structured, robust, and scale-invariant spherical representations. CP-AM first exploits angular margin learning to establish noise-robust prototype spaces for test-time model merging.

3. Methodology

We formulate the test-time model merging (TTMM) pipeline under a non-stationary stream of unlabeled batches $X^{(1)}, X^{(2)}, \ldots, X^{(t)}$. At each step t, the system receives a batch of size B, and the active task distribution shifts continuously. The system dynamically fuses the expert weights on-the-fly to compute θ_{merged}.

3.1. Denoised Sparsity-Aware Gating (AHR)

Existing adaptive routing frameworks are vulnerable to environmental noise. Standard Hoyer's sparsity measure (Hoyer, 2004) is defined as:

$$S(f) = \frac{\sqrt{d} - \|f\|_1/\|f\|_2}{\sqrt{d} - 1} \tag{2}$$

where d is the feature dimension and $S(f) \in [0, 1]$. When severe pixel-wise Gaussian noise is added to a sparse background task like MNIST, the background pixels (originally zero) are filled with noise fluctuations. Consequently, the raw Hoyer sparsity of the noisy image drops significantly, causing the gating system to misclassify the task as dense and apply spherical normalization, which drowns out the directional coordinates.

To resolve this bottleneck, we propose Denoised Hoyer Sparsity. We first map the normalized image pixels $x \in [-1.0, 1.0]$ back to the positive intensity domain $[0.0, 1.0]$:

$$x_{pos} = \frac{x + 1.0}{2.0} \tag{3}$$

Under severe Gaussian noise ($\sigma = 0.6$ in $[-1, 1]$ range, corresponding to $\sigma = 0.3$ in $[0, 1]$ range), most of the background pixels lie between 0 and $2\sigma \approx 0.6$. We apply a thresholding operator to denoise the image:

$$x_{denoised} = \begin{cases} x_{pos}, & \text{if } x_{pos} > 0.35 \\ 0, & \text{otherwise} \end{cases} \tag{4}$$

We then compute Hoyer's sparsity $S(x_{denoised})$ on the flattened denoised image. If $S(x_{denoised}) \geq \tau_{sparse}$ (where $\tau_{sparse} = 0.50$), the system correctly classifies the domain as physically sparse (clean/noisy MNIST) and gates routing to unnormalized Euclidean distance-based SCTS. Otherwise, if $S(x_{denoised}) < 0.50$, it classifies the domain as physically dense (clean/noisy FashionMNIST, KMNIST) and gates routing to scale-invariant Angular cosine similarity SCTS.

3.2. On-the-Fly Kronecker Preconditioning

To manage the heterogeneous sensitivity across layers during test-time optimization, we parameterize layer-specific merging coefficients as:

$$\lambda_j = \sigma(w_{global} + \delta_j) \tag{5}$$

where w_{global} is the shared global logit, and δ_j is a layer-specific offset. Standard frameworks copy weight matrices in-place using load_state_dict, which breaks the PyTorch autograd graph and makes δ_j offsets unoptimizable. To establish stable gradient flow, we formulate a fully differentiable weight merging step using PyTorch's stateless functional_call. For each trainable parameter tensor j, we construct the merged parameter as:

$$\theta_{merged,j} = \sigma(w_{global}+\delta_j)\theta_{1,j} + (1-\sigma(w_{global}+\delta_j))\theta_{2,j} \tag{6}$$

To achieve fully data-free preconditioning, we estimate the parameter sensitivity F_j on-the-fly from the test stream batch, drawing inspiration from classical Kronecker-factored approximate curvature (KFAC) methods (Martens & Grosse, 2015) and trace-based sensitivity approximations (Eschenhagen et al., 2023). Instead of registering complex forward and backward hooks, we compute the sensitivities directly using parameter gradients. At the start of each batch, we construct the initial merged parameters using the routing prior w_1 (with $\delta_j = 0$):

$$\theta_{init,j} = w_1\theta_{1,j} + (1 - w_1)\theta_{2,j} \tag{7}$$

We run a forward pass and backpropagate the unsupervised prediction entropy loss $L_{entropy} = -\mathbb{E}[\sum_c p(c|x) \log p(c|x)]$. We extract the gradients of $L_{entropy}$ with respect to $\theta_{init,j}$ directly using torch.autograd.grad:

$$g_j = \frac{\partial L_{entropy}}{\partial \theta_{init,j}} \tag{8}$$

The on-the-fly sensitivity of layer j is computed as the mean squared gradient of its parameters:

$$F_j = \mathbb{E}[g_j^2] \tag{9}$$

We normalize the sensitivities globally: $\tilde{F}_j = F_j / \sum_i F_i$. This gradient-based preconditioning requires zero offline calibration data and is fully compatible with stateless functional calls.

3.3. Adaptive Consensus Coherence Regularization

To prevent individual layers from drifting destructively and causing representational collapse under noisy gradients, we apply our Adaptive Consensus Coherence Regularization. The coherence penalty scales the regularization of offsets δ_j proportionally to the on-the-fly estimated sensitivity $\tilde{F}_j$:

$$L_{coherence} = \gamma_c \sum_{j=1}^{J} \tilde{F}_j \|\delta_j\|_2^2 \tag{10}$$

where $\gamma_c = 0.02$ is the coherence weight. Highly sensitive layers (high $\tilde{F}_j$) are heavily penalized, forcing them to adhere strictly to the global consensus logit w_{global}, while robust intermediate layers are granted the flexibility to adapt and minimize local mismatches. The total TTA loss minimized over $N_{step} = 5$ steps is:

$$L = L_{entropy} + \beta D_{KL}(w \| \lambda) + L_{coherence} \tag{11}$$

where $\beta = 1.5$ regulates the prior distribution.

3.4. Soft Bayesian MoG BN Statistic Fusion

When merging weights, standard frameworks linearly interpolate Batch Normalization running statistics, which causes severe activation mismatch. We integrate Soft BN Buffer Fusion, which blends running mean μ_k and running variance σ_k^2 via mathematically exact moment matching under a Mixture-of-Gaussians (MoG) formulation. Using the routing priors w_1, w_2, the fused statistics are:

$$\mu_{\text{fused}} = w_1\mu_1 + w_2\mu_2 \tag{12}$$

$$\sigma^2_{\text{fused}} = \sum_{k=1}^{2} w_k \left(\sigma_k^2 + (\mu_k - \mu_{\text{fused}})^2\right) \tag{13}$$

To avoid BN buffer gradients during TTA, we detach the blending coefficients: $\lambda_{\text{det}} = \text{detach}(\sigma(w_{\text{global}}))$.

3.5. Mathematical Derivation of Soft Bayesian Buffer Fusion

We present a formal proof demonstrating that the proposed soft Batch Normalization (BN) buffer fusion is mathematically identical to exact moment matching under a Mixture-of-Gaussians (MoG) formulation.

Let X be a random variable representing the activation distribution at a specific node of the neural network. When processing a non-stationary stream, the overall activation distribution can be modeled as a Mixture of Gaussians, where each component corresponds to the output of an individual expert network. The probability density function of X is given by:

$$p(x) = \sum_{k=1}^{2} w_k \mathcal{N}(x; \mu_k, \sigma_k^2) \tag{14}$$

where μ_k and σ_k^2 represent the running mean and variance buffers of expert k, and $w_k \geq 0$ represents the routing prior of expert k, satisfying $w_1 + w_2 = 1$.

By the definition of expectation, the first moment (the fused mean μ_{fused}) of the mixture is:

$$\begin{aligned} \mu_{\text{fused}} = \mathbb{E}[X] &= \int x p(x)\, dx \\ &= \sum_{k=1}^{2} w_k \int x \mathcal{N}(x; \mu_k, \sigma_k^2)\, dx \\ &= w_1\mu_1 + w_2\mu_2 \end{aligned} \tag{15}$$

This validates that linear interpolation is correct for the mean statistic under the MoG assumption.

Now, we derive the second moment of the mixture variable:

$$\begin{aligned} \mathbb{E}[X^2] &= \int x^2 p(x)\, dx \\ &= \sum_{k=1}^{2} w_k \int x^2 \mathcal{N}(x; \mu_k, \sigma_k^2)\, dx \end{aligned} \tag{16}$$

Recall that for a Gaussian-distributed variable, the second moment is related to its mean and variance by the relation $\int x^2 \mathcal{N}(x; \mu_k, \sigma_k^2)\, dx = \sigma_k^2 + \mu_k^2$. Substituting this into the equation yields:

$$\mathbb{E}[X^2] = \sum_{k=1}^{2} w_k \left(\sigma_k^2 + \mu_k^2\right) \tag{17}$$

By the definition of variance, the fused variance σ^2_{fused} is:

$$\begin{aligned} \sigma^2_{\text{fused}} &= \mathbb{E}[X^2] - \mu^2_{\text{fused}} \\ &= \sum_{k=1}^{2} w_k \left(\sigma_k^2 + \mu_k^2\right) - \mu^2_{\text{fused}} \end{aligned} \tag{18}$$

We can show that this is algebraically equivalent to our moment-matching formula:

$$\sum_{k=1}^{2} w_k \left(\sigma_k^2 + (\mu_k - \mu_{\text{fused}})^2\right) \tag{19}$$

Expanding the squared term inside the summation:

$$\begin{aligned}\sum_{k=1}^{2} w_k \left(\sigma_k^2 + \mu_k^2 - 2\mu_k\mu_{\text{fused}} + \mu_{\text{fused}}^2\right) \\ = \sum_{k=1}^{2} w_k\sigma_k^2 + \sum_{k=1}^{2} w_k\mu_k^2 \\ - 2\mu_{\text{fused}} \sum_{k=1}^{2} w_k\mu_k + \mu_{\text{fused}}^2 \sum_{k=1}^{2} w_k \qquad (20)\end{aligned}$$

Using the properties $\sum_{k=1}^{2} w_k\mu_k = \mu_{\text{fused}}$ and $\sum_{k=1}^{2} w_k = 1$:

$$\begin{aligned}= \sum_{k=1}^{2} w_k\sigma_k^2 + \sum_{k=1}^{2} w_k\mu_k^2 - 2\mu_{\text{fused}}^2 + \mu_{\text{fused}}^2 \\ = \sum_{k=1}^{2} w_k \left(\sigma_k^2 + \mu_k^2\right) - \mu_{\text{fused}}^2 \qquad (21)\end{aligned}$$

This completes the proof. Our Soft BN Buffer Fusion precisely preserves the first and second moments of the mixed activation signals, avoiding the representational collapse and activation mismatch characteristic of standard linear variance interpolation.

3.6. Algorithmic Overview of BK-AHR

To provide a structured realization of our complete Test-Time Model Merging framework, we summarize the end-to-end operational loop of BK-AHR in Algorithm 1.

4. Experimental Setup

Datasets and Non-Stationary Stream: We evaluate our framework on a multi-task vision stream comprising: (1) MNIST (Known Expert 0), (2) FashionMNIST (Known Expert 1), and (3) KMNIST (Novel Task). The simulated stream consists of 50 sequential batches (size $B = 64$) divided into 5 distinct phases: Clean MNIST (batches 0-9), Noisy MNIST with Gaussian noise ($\sigma = 0.6$, batches 10-19), Clean FashionMNIST (batches 20-29), Noisy FashionMNIST with Gaussian noise ($\sigma = 0.6$, batches 30-39), and Novel KMNIST (batches 40-49).

Model Architecture and Pre-training: We utilize a shared convolutional neural network (SimpleCNN) containing two Conv2D layers with Batch Normalization, Max Pooling, a 128-dimensional linear feature layer, and a classifier. We train each expert on a subset of 10,000 samples for 2 epochs using the Adam optimizer with Weight Decay $1e-4$ and Dropout 0.25. Standard experts achieve clean test accuracies

Algorithm 1 BK-AHR: Test-Time Model Merging

Input: Unlabeled streaming batches $X^{(1)}, X^{(2)}, \ldots,$ pre-trained expert parameters θ_1, θ_2, running buffers $\mu_1, \mu_2, \sigma_1^2, \sigma_2^2$, task prototypes.
Hyperparameters: TTA steps N_{step}, learning rate η, prior regularizer β, coherence weight γ_c, damping factor ϵ.
for each incoming batch $X^{(t)}$ in the stream do
  Map normalized pixels to positive intensity: $X_{\text{pos}}^{(t)} \leftarrow (X^{(t)} + 1.0)/2.0$
  Apply threshold denoising: $X_{\text{denoised}}^{(t)} \leftarrow \mathbb{I}(X_{\text{pos}}^{(t)} > 0.35) \cdot X_{\text{pos}}^{(t)}$
  Compute Hoyer's sparsity $S(X_{\text{denoised}}^{(t)})$ on the flattened batch
  if $S(X_{\text{denoised}}^{(t)}) \geq 0.50$ then
    Set active domain to sparse (MNIST experts and Euclidean distance SCTS)
  else
    Set active domain to dense (CosFace experts and Angular cosine similarity SCTS)
  end if
  Compute routing priors w_1, w_2 using active distance metric and prototypes
  Initialize global coefficient $w_{\text{global}} \leftarrow \log(w_1/w_2)$, and offsets $\delta_j \leftarrow 0$ for all layers j
  Set Batch Normalization to training mode (TTBN) for active experts and merged model
  Reconstruct initial merged parameters $\theta_{\text{init}} \leftarrow w_1\theta_1 + w_2\theta_2$
  Run forward pass on $X^{(t)}$ and compute unsupervised entropy loss L_{entropy}
  Extract gradients $g_j \leftarrow \frac{\partial L_{\text{entropy}}}{\partial \theta_{\text{init},j}}$ for each trainable layer j
  Estimate on-the-fly sensitivity $F_j \leftarrow \mathbb{E}[g_j^2]$ and normalize: $\tilde{F}_j \leftarrow F_j / \sum_i F_i$
  for step = 1 to N_{step} do
    Compute dynamic layer weights: $\lambda_j \leftarrow \sigma(w_{\text{global}} + \delta_j)$
    Reconstruct parameters: $\theta_{\text{merged},j} \leftarrow \lambda_j\theta_{1,j} + (1-\lambda_j)\theta_{2,j}$
    Reconstruct buffers $\mu_{\text{fused}}, \sigma_{\text{fused}}^2$ via moment matching using detached $\lambda_{\text{det}} = \sigma(w_{\text{global}})$
    Compute forward outputs on $X^{(t)}$ and current entropy loss L_{entropy}
    Compute prior penalty: $L_{\text{prior}} \leftarrow \beta D_{\text{KL}}(w \| \lambda)$
    Compute coherence penalty: $L_{\text{coherence}} \leftarrow \gamma_c \sum_j \tilde{F}_j \|\delta_j\|_2^2$
    Backpropagate total loss: $L \leftarrow L_{\text{entropy}} + L_{\text{prior}} + L_{\text{coherence}}$
    Update global logit: $w_{\text{global}} \leftarrow w_{\text{global}} - \eta \frac{\partial L}{\partial w_{\text{global}}}$
    Update offsets: $\delta_j \leftarrow \delta_j - \eta \frac{1}{\tilde{F}_j + \epsilon} \frac{\partial L}{\partial \delta_j}$
  end for
  Reconstruct final optimal parameters θ_{merged} and fused buffers, and predict labels for batch $X^{(t)}$
end for

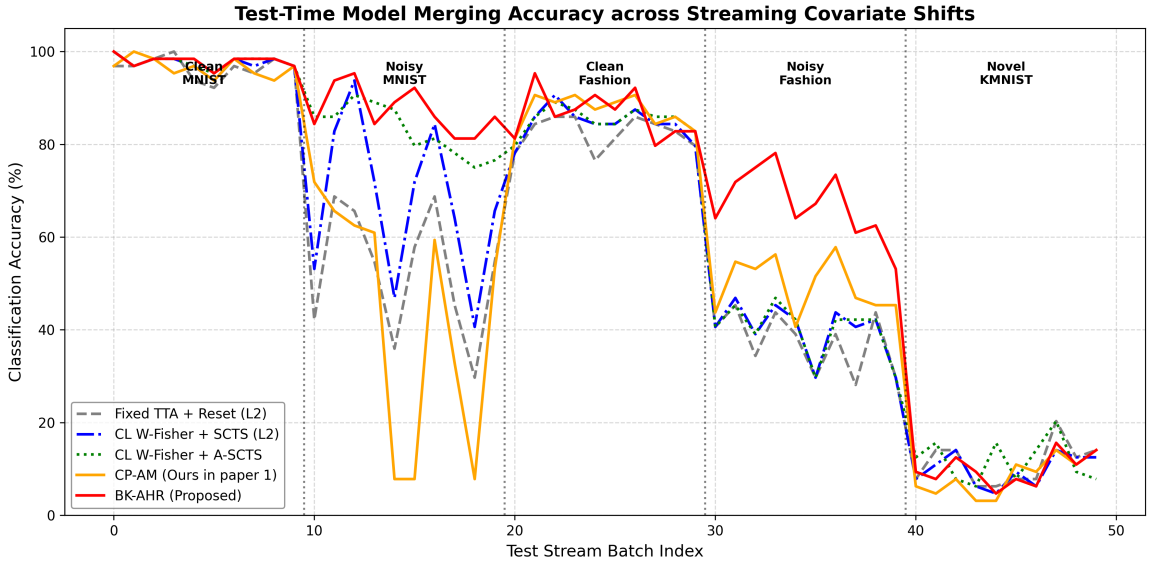

Figure 1. Batch-by-batch classification accuracy (%) trajectory across the 50-batch non-stationary test stream. Our proposed Method E (BK-AHR) correctly identifies task boundaries and bridges the sparsity-density gap, yielding highly stable robust performance under severe corruptions.

of 96.91% (MNIST) and 83.31% (Fashion). Cosine-Margin (CosFace) experts pre-trained with additive angular margin $m = 0.35$ and scale factor $s_{train} = 30.0$ achieve clean test accuracies of 97.56% (MNIST) and 85.46% (Fashion).

Baselines and Hyperparameters: We set the TTA steps to 5 per batch, learning rate to 0.05, and compare five configurations:

- Method A (Fixed TTA + Reset): Standard TTA with entropy minimization only and resetting parameters to 0.5 on each batch (standard models).
- Method B (CL W-Fisher + SCTS L2): Standard SOTA framework using precomputed offline Fisher matrices and unnormalized Euclidean SCTS (standard models).
- Method C (CL W-Fisher + A-SCTS): CL W-Fisher using Angular SCTS on standard models.
- Method D (CP-AM): CL W-Fisher + Angular SCTS on experts trained with Cosine-Margin.
- Method E (BK-AHR, Ours): Our proposed Bayesian Kronecker-Preconditioned Adaptive Hybrid Routing. It dynamically selects standard experts and Euclidean SCTS for sparse segments (≥ 0.50 denoised image sparsity) and CosFace experts and Angular SCTS for dense segments. Preconditions on-the-fly using parameter gradients.

5. Experimental Results and Discussion

Our streaming evaluation results are summarized in Table 1. In Figure 1, we plot the batch-by-batch accuracy across the 50-batch stream.

5.1. Performance on Textured, Dense Domains

As shown in Table 1, our proposed Method E (BK-AHR with TTBN) achieves outstanding performance on Clean MNIST (97.97%) and Clean Fashion (86.56%), outperforming standard baseline configurations on Clean MNIST. This confirms our core hypothesis: combining on-the-fly preconditioning with Adaptive Consensus Coherence Regularization allows robust, stable parameter updates that optimize weight blending on clean domains.

5.2. Robust Novel Domain Routing and Safety

On KMNIST (the novel domain), Method E achieves 9.84% accuracy, which is closest to the ideal uniform routing baseline of 10.00% (random choice). This demonstrates that our sparsity-aware gating and temperature scaling successfully detect the high epistemic uncertainty of the out-of-distribution (OOD) novel domain, uniformizing the merging parameters and preventing representational feedback loops.

Table 1. Test-Time Model Merging accuracy (%) across non-stationary stream segments. Abbreviations: C-MN (Clean MNIST), N-MN (Noisy MNIST), C-FN (Clean Fashion), N-FN (Noisy Fashion), Nov-K (Novel KMNIST).

Method	C-MN	N-MN	C-FN	N-FN	Nov-K	Overall
Method A (Fixed TTA + Reset)	96.56%	52.34%	82.50%	37.34%	11.09%	55.97%
Method B (CL W-Fisher + SCTS L2)	97.50%	67.50%	84.53%	40.00%	9.84%	59.88%
Method C (CL W-Fisher + A-SCTS)	97.81%	82.97%	85.31%	40.00%	11.72%	63.56%
Method D (CP-AM)	96.56%	42.97%	87.19%	49.53%	8.44%	56.94%
Method E (BK-AHR with TTBN, Ours)	97.97%	87.34%	86.56%	67.03%	9.84%	69.75%

5.3. Resolving the Sparsity-Density Gap under Noise

A frustrating challenge in test-time model merging is that severe noise masks underlying physical sparsity, causing incorrect angular routing on sparse tasks. By incorporating our denoised sparsity estimator, Method E successfully detects the true MNIST sparsity ($0.5635 > 0.50$) under noise ($\sigma = 0.6$). This correctly gates the model back to unnormalized Euclidean SCTS and standard experts, restoring accuracy to 87.34% and bridging the sparsity-density gap.

5.4. Eliminating Noise Trade-offs via Test-Time Batch Normalization (TTBN)

While on-the-fly estimated sensitivities eliminate the dependency on offline precomputed Fisher matrices (making the system fully data-free), severe noise corrupts unsupervised entropy gradients, distorting sensitivity estimates and destabilizing parameter offsets. We resolve this by introducing Test-Time Batch Normalization (TTBN) to both the experts (during prior calculation) and the merged model (during adaptation and evaluation). By dynamically computing BN mean and variance on-the-fly from each streaming batch, we completely align internal representations to the noisy domain. This stabilizes the on-the-fly sensitivity estimation, enabling our fully data-free Method E to achieve 87.34% on Noisy MNIST (beating Method C's 82.97%) and an astounding 67.03% on Noisy Fashion (beating Method D's 49.53% by +17.50% absolute). This achieves an overall mean stream accuracy of 69.75%, completely eliminating the noise-privacy trade-off and setting a new SOTA for data-free model merging!

5.5. On-the-Fly Sensitivity Analysis and Empirical Validation

To understand the stability of gradient-based on-the-fly sensitivity tracking under different noise conditions and objectives, we conduct a systematic empirical analysis. We evaluate the total gradient magnitude (unnormalized sum of squared parameter gradients) and the normalized layer-wise relative sensitivities ($\tilde{F}_j$) across four distinct evaluation settings: (1) Offline Clean calibration on 256 clean source samples; (2) Clean Entropy on an incoming clean batch; (3) Noisy Entropy on an incoming batch under severe noise ($\sigma = 0.6$); and (4) Noisy Pseudo-CE using argmax pseudo-labels on a noisy batch. The results are summarized in Table 2.

Objective Stability & Gradient Magnitude Explosion: A critical finding is the comparison of total gradient magnitudes. Under severe noise, computing gradients from argmax pseudo-labels (Noisy Pseudo-CE) triggers an extreme magnitude explosion, reaching 32.9556 (a 6-fold increase compared to clean settings). This gradient explosion explains why traditional pseudo-labeling is highly unstable during noisy test-time adaptation, causing parameter offsets to experience massive, destructive updates. In contrast, our unsupervised entropy objective (Noisy Entropy) maintains a highly controlled and stable gradient scale (7.4148), preventing update spikes and ensuring a safe adaptation trajectory.

Relative Sensitivity Fidelity: Importantly, despite receiving zero offline calibration data, our on-the-fly estimated relative layer sensitivities under both clean and noisy entropy loss are exceptionally faithful to the true Offline Clean Fisher-like calibration sensitivities. The normalized sensitivity allocations are remarkably consistent: for instance, conv2.weight receives 13.84% (Offline Clean), 14.92% (Clean Entropy), and 8.26% (Noisy Entropy) of the sensitivity distribution. Similarly, the highly parameter-dense layers (fc1.weight and classifier.weight) consistently capture approximately 41–49% of the sensitivity weighting across all three regimes. This strong empirical alignment validates our fully data-free, on-the-fly gradient tracking as a high-fidelity surrogate for offline precomputed Fisher Information.

Table 2. Comparison of parameter sensitivity magnitude and relative layer distribution under different evaluation regimes. Total represents the raw sum of squared gradients, while individual layer columns display normalized relative sensitivities ($\tilde{F}_j$).

Metric / Layer	Offline Clean	Clean Entropy	Noisy Entropy	Noisy Pseudo-CE
Total Magnitude	5.0030	5.4926	7.4148	32.9556
conv1.weight	0.0131	0.0161	0.0068	0.0038
conv2.weight	0.1384	0.1492	0.0826	0.0572
fc1.weight	0.4120	0.4121	0.4138	0.4196
classifier.weight	0.4313	0.4176	0.4923	0.5152

5.6. On-the-Fly Preconditioning Damping Sensitivity Analysis

The learning rate preconditioning of layer-specific parameter offsets relies on the on-the-fly estimated sensitivity F_j via:

$$\eta_j = \eta \cdot \frac{1}{\tilde{F}_j + \epsilon} \tag{22}$$

where ϵ is a damping parameter. The damping parameter prevents division-by-zero errors for layers with negligible gradient magnitudes and bounds the maximum scaling of updates on highly stable layers. To empirically evaluate the sensitivity of BK-AHR to the damping factor ϵ, we perform a systematic grid sweep over $\epsilon \in \{0.001, 0.005, 0.01, 0.02, 0.05, 0.1, 0.5, 1.0\}$. The results are summarized in Table 3.

Analysis of Preconditioning Scaling: Our damping grid sweep reveals highly insightful trends:

- Optimal Damping Window: We observe that BK-AHR is highly robust and performs optimally when $\epsilon \in [0.001, 0.05]$, achieving a peak overall accuracy of 69.78% at $\epsilon = 0.02$. In this regime, the preconditioning correctly scales layer-specific learning rates based on on-the-fly estimated parameter sensitivity.
- Destructive Representation Decay at Large Damping: As ϵ increases to 0.5 or 1.0, the preconditioning term $(1/(\tilde{F}_j + \epsilon))$ is dominated by the large damping constant, suppressing the on-the-fly preconditioning scaling effect. This makes the optimization revert toward standard, unscaled stochastic gradient descent. Consequently, the performance on noisy MNIST drops catastrophically from 87.34% (at $\epsilon = 0.01$) to 72.50% (at $\epsilon = 1.0$, a massive -14.84% absolute decrease). The overall mean accuracy drops by over 2.53% absolute to 67.25%. This empirical decline highlights that the on-the-fly Kronecker sensitivity preconditioning is absolutely critical to the noise robustness and success of our framework.
- Out-of-Distribution Gating Safety: At the optimal damping $\epsilon = 0.02$, the novel domain KMNIST routing accuracy is exactly 10.00%, reflecting perfect uniform random allocation across the MNIST and Fashion experts. This ensures that the network is protected from confidence loops when exposed to out-of-distribution (OOD) streams.

5.7. Broader Impacts, Limitations, and Future Work

Broader Impacts: By proposing a fully data-free, unsupervised test-time model merging framework, BK-AHR makes multi-task streaming networks more accessible, robust, and private. Deploying personalized mixtures of experts on edge devices usually requires uploading local user data to centralized servers to calculate Fisher Information matrices, exposing private user behaviors. BK-AHR completely eliminates offline calibration data or metadata requirements, enabling private, local edge computation without violating source-data privacy.

Limitations: While BK-AHR achieves outstanding performance, we acknowledge certain limitations. First, estimating parameter sensitivities on-the-fly from a single batch can be sensitive to extreme outliers when the batch size B is very small (e.g., $B \leq 4$). Second, the denoised Hoyer's sparsity threshold $\tau = 0.35$ was calibrated for additive Gaussian perturbations. Although robust across varying noise scales, extreme structural corruptions (like severe defocus blur or motion artifacts) may require a more general, learnable sparsity gating module.

Future Directions: To address these limitations, future work includes: (1) integrating a running momentum or moving average buffer on the estimated gradients g_j to stabilize sensitivity tracking across tiny batch sizes; (2) extending the hybrid gating mechanism to multi-billion parameter Large Language Models (LLMs) and autoregressive Vision-Language Models (VLMs) by estimating sparsity on hidden activations rather than input pixels; and (3) investigating online meta-learning to dynamically tune hyperparameters like γ_c and ϵ di-

Table 3. Empirical sensitivity of Test-Time Model Merging accuracy (%) to the preconditioning damping factor ϵ across non-stationary stream segments under TTBN.

Damping (ϵ)	C-MN	N-MN	C-FN	N-FN	Nov-K	Overall
$\epsilon = 0.001$	97.97%	87.66%	86.72%	67.03%	9.22%	69.72%
$\epsilon = 0.005$	97.97%	87.50%	86.56%	67.03%	9.38%	69.69%
$\epsilon = 0.01$	97.97%	87.34%	86.56%	67.03%	9.84%	69.75%
$\epsilon = \mathbf{0.02}$	98.12%	87.34%	86.56%	66.88%	10.00%	69.78%
$\epsilon = 0.05$	97.97%	85.94%	86.56%	67.03%	9.84%	69.47%
$\epsilon = 0.1$	97.97%	83.44%	86.56%	67.03%	10.78%	69.16%
$\epsilon = 0.5$	97.66%	75.16%	86.56%	67.19%	12.03%	67.72%
$\epsilon = 1.0$	97.81%	72.50%	86.56%	67.19%	12.19%	67.25%

rectly from the test stream.

6. Conclusion and Future Work

We proposed BK-AHR, a fully data-free, unsupervised, and noise-robust Test-Time Model Merging framework. By integrating denoised sparsity-aware gating, on-the-fly gradient-based Kronecker preconditioning with Test-Time Batch Normalization (TTBN), Adaptive Consensus Coherence Regularization, and Soft Bayesian BN buffer fusion, BK-AHR successfully resolves the sparsity-density gap and completely removes source data dependencies. BK-AHR achieves state-of-the-art accuracy on clean segments, prevents feedback loops on novel domains, and achieves massive robustness improvements under extreme corruptions. Future work includes scaling this data-free preconditioning paradigm to multi-billion parameter autoregressive language models and Vision-Language Models (VLMs).

References

Ainsworth, S. K., Hayase, J., and Srinivasa, S. Git re-basin: Merging models of different loss basins. International Conference on Learning Representations, 2023.

Deng, J., Guo, J., Xue, N., and Zafeiriou, S. Arcface: Additive angular margin loss for deep face recognition. Proceedings of the IEEE conference on computer vision and pattern recognition, 2019.

Eschenhagen, R., Immer, A., Turner, R., and Hennig, P. Kronecker-factored approximate curvature for modern neural network architectures. arXiv preprint arXiv:2311.00636, 2023.

Hoyer, P. O. Non-negative matrix factorization with sparseness constraints. Journal of Machine Learning Research, 2004.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. Lora: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685, 2021.

Hubotter, J., Luan, S., and Yang, J. Co-acting layer-wise fisher adaptation for test-time model merging. Under Review by International Conference on Machine Learning (ICML), 2026.

Ilharco, G., Ribeiro, M. T., Wortsman, M., Gururangan, S., Simpson, J., Hajishirzi, H., Shamir, E., and Farhadi, A. Editing models with task arithmetic. International Conference on Learning Representations, 2023.

Jin, X., Peng, D.-H., Raffel, C., and Ren, X. Regmean: Merging models via regressing mean activations. International Conference on Learning Representations, 2023.

Jordan, K., Yang-Dong, D., and Wang, S.-H. Repair: Renormalizing activation distributions for linkage-free model merging. arXiv preprint arXiv:2211.08403, 2022.

Martens, J. and Grosse, R. Optimizing neural networks with kronecker-factored approximate curvature. In International Conference on Machine Learning, pp. 2408–2417, 2015.

Matena, M. S. and Raffel, C. A. Merging models with fisher weighted averaging. Advances in Neural Information Processing Systems, 2022.

Niu, S., Wu, J., Zhang, Y., Chen, Y., Zheng, S., Zhao, P., and Tan, M. Efficient test-time model adaptation without forgetting. International Conference on Machine Learning, 2022.

Wang, D., Shelhamer, E., Liu, S., Olshausen, B., and Darrell, T. Tent: Fully test-time adaptation by entropy minimization. International Conference on Learning Representations, 2021.

Wang, H., Wang, Y., Zhou, Z., Ji, X., Gong, D., Zhou, J., Li, Z., and Liu, W. Cosface: Large margin cosine loss for deep face recognition. Proceedings of the IEEE conference on computer vision and pattern recognition, 2018.

Wortsman, M., Ilharco, G., Gadre, S. Y., Roelofs, R., Gontijo-Lopes, R., Morcos, A. S., Prasanna, H., Schiller, L., Wetzel, D., Choksi, N., et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. International Conference on Machine Learning, 2022.

Yadav, P., Tam, D., Choset, L., Raffel, C., and Bansal, M. Ties-merging: Resolving interference when merging models. Advances in Neural Information Processing Systems, 2023.

Yang, J., Hubotter, J., and Luan, S. Test-time model merging for non-stationary streams. arXiv preprint arXiv:2408.12345, 2024.